

Introduction to Recommender Systems

Recommender systems are among the most commercially valuable applications of machine learning. While they receive moderate attention in academia, they are foundational to the success of major tech companies.

- **Examples:** Amazon (product suggestions), Netflix (movie picks), food delivery apps (restaurant ideas).
 - **Impact:** For many of these companies, a significant fraction of total sales/consumption is driven by automated recommendations.
-

The Problem Setup: Movie Ratings Example

To understand how these systems work, we use a running example of a movie streaming platform where users rate movies on a scale of **0 to 5 stars**.

The Data:

Imagine a dataset with 4 users and 5 movies.

Movie (Item)	Alice (1)	Bob (2)	Carol (3)	Dave (4)
Love at last	5	5	0	0
Romance forever	5	?	?	0
Cute puppies of love	?	4	0	?
Nonstop car chases	0	0	5	5
Swords vs. Karate	0	0	5	?

- **Observations:**
 - **Alice & Bob:** Prefer romance/soft topics (rated 4 or 5) and dislike action (rated 0).
 - **Carol & Dave:** Prefer action (rated 5) and dislike romance (rated 0).
 - **The Goal:** The system needs to look at the missing ratings (marked with ?) and predict what score the user *would* give if they watched it.
 - **The Action:** Recommend movies that are predicted to receive high ratings (e.g., 5 stars).
-

Formal Notation

To build an algorithm, we need standard notation to describe this data.

- n_u : The number of **users**. (In this example, $n_u = 4$).
- n_m : The number of **movies** (items). (In this example, $n_m = 5$).
- $r(i, j)$: A flag indicating if a rating exists.
 - $r(i, j) = 1$ if user j **has** rated movie i .
 - $r(i, j) = 0$ if user j **has not** rated movie i .
- $y^{(i,j)}$: The actual rating given.
 - This value only exists if $r(i, j) = 1$.
 - Example: $y^{(3,2)} = 4$ (User 2 rated Movie 3 a score of 4).

The Approach

The core task of the recommender system is to predict values for the empty cells where $r(i, j) = 0$.

In the upcoming lessons, we will develop an algorithm to solve this, starting with an assumption that we have specific **features** about the movies (e.g., knowing exactly how much "Action" or "Romance" is in a film) and then evolving to techniques that work even without that data.